1.8.28

EE25BTECH11011 - Navya Banavathu

September 2,2025

Question

Find the equation of the set of the points P such that its distances from the points $A(3, 4, -5)$ and $B(-2, 1, 4)$ are equal.

Solution

Let the required point be denoted by the vector $\mathbf{X} \in \mathbb{R}^3$. The condition that $\mathbf{X}$ is equidistant from $\mathbf{A}$ and $\mathbf{B}$ can be written using the norm (distance) as

$$\|\mathbf{X} - \mathbf{A}\| = \|\mathbf{X} - \mathbf{B}\|. \tag{1}$$

Squaring both sides (to remove the square root) gives

$$\|\mathbf{X} - \mathbf{A}\|^2 = \|\mathbf{X} - \mathbf{B}\|^2. \tag{2}$$

Using $\|\mathbf{u}\|^2 = \mathbf{u}^T\mathbf{u}$, expand both sides:

$$(\mathbf{X} - \mathbf{A})^T(\mathbf{X} - \mathbf{A}) = (\mathbf{X} - \mathbf{B})^T(\mathbf{X} - \mathbf{B}). \tag{3}$$

Expanding the products yields

$$\mathbf{X}^T\mathbf{X} - 2\mathbf{A}^T\mathbf{X} + \mathbf{A}^T\mathbf{A} = \mathbf{X}^T\mathbf{X} - 2\mathbf{B}^T\mathbf{X} + \mathbf{B}^T\mathbf{B}. \tag{4}$$

Solution

The $\mathbf{X}^T\mathbf{X}$ terms cancel, leaving a linear equation in $\mathbf{X}$:

$$-2\mathbf{A}^T\mathbf{X} + \mathbf{A}^T\mathbf{A} = -2\mathbf{B}^T\mathbf{X} + \mathbf{B}^T\mathbf{B}. \tag{5}$$

Rearrange to isolate the inner product with $\mathbf{X}$:

$$2(\mathbf{B} - \mathbf{A})^T\mathbf{X} = \mathbf{B}^T\mathbf{B} - \mathbf{A}^T\mathbf{A}. \tag{6}$$

Now compute the two vector/scalar pieces on the right:

$$\mathbf{B} - \mathbf{A} = \begin{pmatrix} -2 \\ 1 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 4 \\ -5 \end{pmatrix} = \begin{pmatrix} -5 \\ -3 \\ 9 \end{pmatrix}, \tag{7}$$

$$\mathbf{B}^T\mathbf{B} = (-2)^2 + 1^2 + 4^2 = 21, \tag{8}$$

$$\mathbf{A}^T\mathbf{A} = 3^2 + 4^2 + (-5)^2 = 50. \tag{9}$$

Solution

Substitute these values into the linear equation:

$$2\begin{pmatrix}-5 & -3 & 9\end{pmatrix}\mathbf{X} = 21 - 50 = -29. \tag{10}$$

Simplify the left-hand side:

$$\begin{pmatrix}-10 & -6 & 18\end{pmatrix}\mathbf{X} = -29. \tag{11}$$

Hence the equation of the plane is given by: $\boxed{\begin{pmatrix}-10 & -6 & 18\end{pmatrix}\mathbf{X} = -29}$

Code (C)

Part 1

```c
#include <stdio.h>
//dimension
#define D 3
//compute C = U - V (vector subtraction)
void vec_sub(const int U[D], const int V[D], int C[D]) {
    for (int i = 0; i < D; ++i) C[i] = U[i] - V[i];
}
// compute dot = U^T * V (inner product)
int dot(const int U[D], const int V[D]) {
    int s = 0;
    for (int i = 0; i < D; ++i) s += U[i] * V[i];
    return s;
}
// multiply vector by scalar: R = k * V
void scalar_mul(int k, const int V[D], int R[D]) {
    for (int i = 0; i < D; ++i) R[i] = k * V[i];
}
```

Code (C)

Part 2

```c
//print row-vector like [a b c]
void print_rowvec(const int V[D]) {
    printf("[");
    for (int i = 0; i < D; ++i) {
        printf("%d", V[i]);
        if (i < D-1) printf(" ");
    }
    printf("]");
}
int main(void) {
    // Given points (column vectors) A and B
    int A[D] = {3, 4, -5};
    int B[D] = {-2, 1, 4};
    // Step 1: compute B - A
    int BminusA[D];
    vec_sub(B, A, BminusA);
    // Step 2: compute left-hand row vector = 2 (B - A)^T (we
```

Code (C)

Part 3

```
    int left_row[D];
    scalar_mul(2, BminusA, left_row);

 // Step 3: compute RHS scalar = B^T B - A^T A (dot products)
    int BB = dot(B, B);
    int AA = dot(A, A);
    int rhs = BB - AA;

// Output (matrix-theory style)
 printf("Using matrix theory: 2(B - A)^T X = B^T B - A^T A\n\n");
    printf("B - A (column vector) = ");
    print_rowvec(BminusA); // prints as row for compactness
    printf(" (i.e. column vector with these entries)\n");

    printf("Left row vector 2(B - A)^T = ");
    print_rowvec(left_row);
    printf("\n");
```

Code (C)

Part 4

```
printf("RHS scalar B^T B - A^T A = %d - %d = %d\n\n", BB, AA,
    rhs);

// Print final plane equation in standard algebraic form
// left_row corresponds to coefficients [a b c] so equation is a
     x + b y + c z = rhs
printf("Hence the equation of the plane (matrix form) is:\n");
printf(" ");
print_rowvec(left_row);
printf(" X = %d\n\n", rhs);

// also print conventional algebraic equation
printf("Or in coordinate form:\n");
printf(" %dx + %dy + %dz = %d\n", left_row[0], left_row[1],
    left_row[2], rhs);
return 0;
}
```

Python Code:Import and Define points

Part 1

```
import ctypes
import numpy as np

# Load compiled shared library
lib = ctypes.CDLL("./plane.so") # use "plane.dll" on Windows

D = 3
IntArray3 = ctypes.c_int * D

# Bind functions
lib.vec_sub.argtypes = [IntArray3, IntArray3, IntArray3]
lib.dot.argtypes = [IntArray3, IntArray3]
lib.dot.restype = ctypes.c_int
lib.scalar_mul.argtypes = [ctypes.c_int, IntArray3, IntArray3]
```

Python Code

Part 2

```
def vec_sub(U, V):
    U_arr, V_arr, C_arr = IntArray3(*U), IntArray3(*V), IntArray3
        ()
    lib.vec_sub(U_arr, V_arr, C_arr)
    return list(C_arr)

  def dot(U, V):
    return lib.dot(IntArray3(*U), IntArray3(*V))

def scalar_mul(k, V):
    V_arr, R_arr = IntArray3(*V), IntArray3()
    lib.scalar_mul(k, V_arr, R_arr)
    return list(R_arr)

if __name__ == "__main__":
    A = [3, 4, -5]
    B = [-2, 1, 4]
```

Python Code

Part 3

```
 BminusA = vec_sub(B, A)
   left_row = scalar_mul(2, BminusA)
   rhs = dot(B, B) - dot(A, A)

print("Using C library through ctypes:")
   print("B - A =", BminusA)
   print("Left row vector 2(B - A)^T =", left_row)
   print("RHS scalar =", rhs)
   print(f"Equation of plane: {left_row[0]}x + {left_row[1]}y +
       {left_row[2]}z = {rhs}")
```

Python Code: Plotting

Part 4

```
 fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

# Define grid for plane
x_vals = np.linspace(-6, 6, 20)
y_vals = np.linspace(-6, 6, 20)
X, Y = np.meshgrid(x_vals, y_vals)
Z = (rhs - coeff[0]*X - coeff[1]*Y) / coeff[2]

# Plot the plane
ax.plot_surface(X, Y, Z, alpha=0.5, color="cyan")
 # Plot points A, B, and midpoint M
ax.scatter(*A, color="red", s=50, label="A(3,4,-5)")
ax.scatter(*B, color="blue", s=50, label="B(-2,1,4)")
ax.scatter(*M, color="black", s=50, label="Midpoint M")
```

Python Code: Plotting

Part 5

```
# Plot AB line
ax.plot([A[0], B[0]], [A[1], B[1]], [A[2], B[2]], "k--", label="
    AB")

# Labels
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
ax.set_zlabel("Z axis")
ax.legend()

plt.title("Perpendicular Bisector Plane of A and B (Matrix Theory
    )")
plt.savefig("fig2.png", dpi=300, bbox_inches="tight") # Save the
    figure
plt.show()
```

Graphical Representation

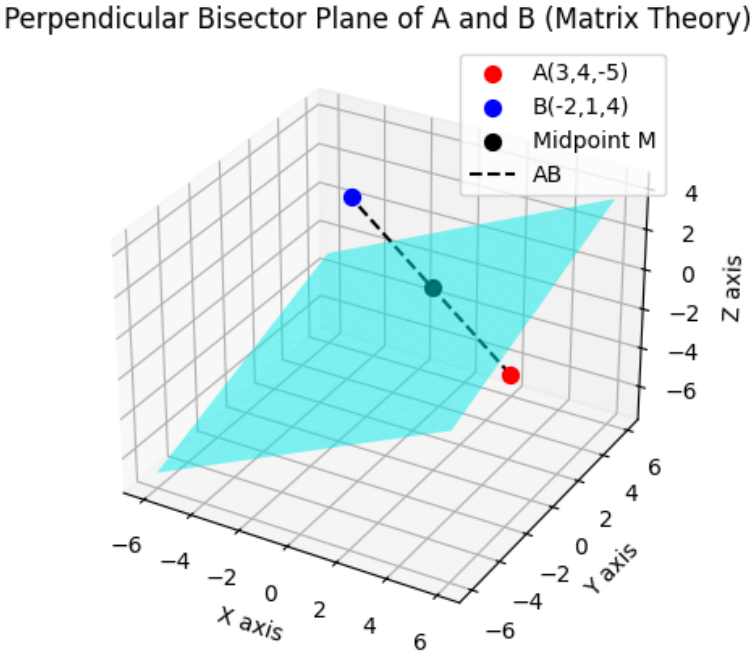

Figure: locus of point P